

Plan: Harden user creation route

Tighten the POST /users implementation in users.route.ts by adding stricter request validation, abuse protection, and security-relevant logging while preserving the existing 409 USER_EXISTS contract. The recommended approach is to update the route and its tests first, add the missing runtime dependencies in package.json, and treat the shared-logger path as an explicit implementation check because the repo instructions require structured logging but no logger file is currently discoverable.

Steps

1. Confirm the execution surface and preserve current API semantics: keep POST /users in users.route.ts and retain the 409 USER_EXISTS duplicate-email behavior instead of the anti-enumeration response. This locks the acceptance criteria before any code changes.
2. Add validation dependencies and align the route with repo conventions: update package.json to add Zod for request-body validation, then introduce a schema for name/email normalization and constraints that rejects blank strings, malformed emails, and clearly unreasonable lengths. This step blocks the route refactor because the handler should parse through the schema rather than through ad hoc field checks.
3. Refactor createUserHandler in users.route.ts to use schema-driven parsing and normalization while preserving the current response shape for success and duplicate-email conflicts. Reuse the existing user creation flow, but move validation responsibility to the schema so invalid payloads fail consistently and tests can target stricter rules.
4. Add abuse protection to the route registration path: add express-rate-limit to package.json and apply a focused limiter to the POST /users route in usersRouter. Scope the limiter to this endpoint only and choose conservative defaults suitable for a demo route backed by an in-memory Map.
5. Add structured security logging around duplicate attempts and successful creation. First verify whether a shared logger module already exists at the repo-standard location; if not, create or agree on the minimal shared logger module before wiring logs into the handler. Log only safe structured fields such as userId or a one-way email hash, and do not log raw email/password/body content.
6. Update users.route.test.ts to match the hardened behavior. Keep the existing happy path and duplicate-email assertions, add explicit invalid-input cases for whitespace-only name and malformed email, and add targeted checks for any changed validation payload shape that results from schema parsing. If the rate limiter is implemented directly in the router rather than the handler, cover it with a narrow integration-style test only if the current test harness already supports router-level execution; otherwise defer rate-limit verification to a follow-up app-level test.
7. Run focused verification: execute the users.route test file or the smallest Vitest scope available, then run the full test suite only if the narrow check passes and no faster validation is available. If dependency or logger-path issues surface, resolve those locally before broadening scope.

Relevant files

- d:\Github - Manish\gh_copilot_afternoon_25_May_2026\node-demo\users.route.ts — primary implementation target; currently contains inline validation, duplicate-email handling, and route registration.
- d:\Github - Manish\gh_copilot_afternoon_25_May_2026\node-demo\users.route.test.ts — update coverage for stricter validation and preserved duplicate behavior.
- d:\Github - Manish\gh_copilot_afternoon_25_May_2026\node-demo\package.json — add Zod and express-rate-limit dependencies required by the hardening changes.
- d:\Github - Manish\gh_copilot_afternoon_25_May_2026\node-demo.github\copilot-instructions.md — source of repo conventions to follow: Zod-based validation, structured logging, and Vitest tests.
- d:\Github - Manish\gh_copilot_afternoon_25_May_2026\node-demo.github\instructions\backend.instructions.md — confirms validation, logging, and rate-limiting expectations for backend TypeScript files.

- d:\Github - Manish\gh_copilot_afternoon_25_May_2026\node-demo.github\instructions\test.instructions.md
— test constraints for deterministic Vitest coverage.

Verification

1. Run the narrowest Vitest check for `users.route.test.ts` and confirm the existing happy path and duplicate-email tests still pass.
2. Add and run assertions for invalid payloads: blank name, blank email, malformed email, and normalized email duplicates.
3. If the rate limiter is testable within the current harness, verify repeated POST `/users` requests eventually return the configured throttle status without breaking earlier tests.
4. Confirm structured logs do not include raw email addresses, passwords, or full request bodies.

Decisions

- Included: stronger input validation, route-level rate limiting, and security logging for registration events.
- Excluded: changing duplicate-email handling to a generic anti-enumeration response; the plan preserves the current 409 `USER_EXISTS` contract.
- Assumption: creating or wiring a shared logger may be necessary because no logger file is currently discoverable in the workspace.

Further Considerations

1. If the team wants full compliance with the layered architecture in the repo instructions, a follow-up plan should move user creation logic out of the route file into controller/service layers instead of growing `users.route.ts` further.
2. If the route is later exposed beyond demo use, add app-level protections such as Helmet, centralized error handling, and request IDs rather than keeping security controls limited to this single route.